CENG3522-Applied Machine Learning

AI-Human Face Distinction

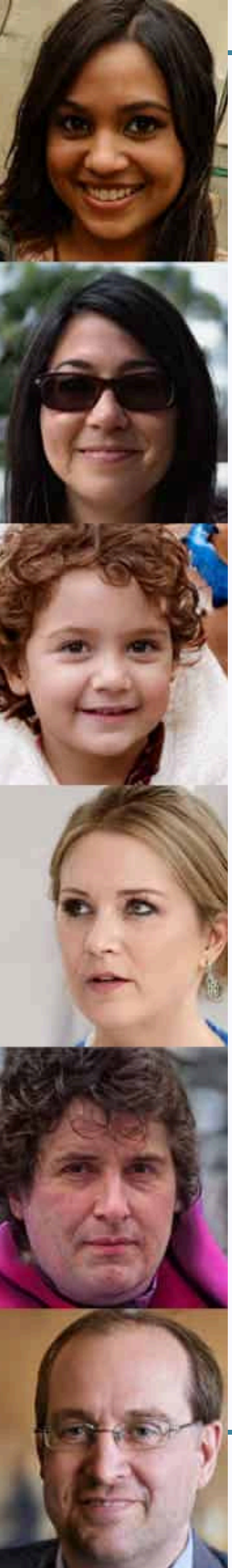
TEAM MEMBERS

THE AWESOME PEOPLE BEHİND THİS PROJECT

MERVE KALDIRIM

HALİL EFE ÇAKMAK

RECEP EFEKAN BAS



PROBLEM STATEMENT AND MOTIVATION

With the rapid advancement of generative models such as GANs, AI-generated images have become increasingly realistic and harder to distinguish from authentic photographs. This progress brings significant challenges in various domains, including journalism, social media, security, and digital forensics, where verifying the authenticity of visual content is critical.

The main problem addressed in this project is the automatic detection of AI-generated (synthetic) images to help prevent the spread of misleading or manipulated visual content. The goal is to develop and compare different detection approaches that can reliably classify images as either real or fake, using both traditional machine learning techniques and more advanced detection models.

This study is motivated by the need for robust, efficient, and scalable tools that can support individuals and organizations in verifying the credibility of digital images. To address this challenge comprehensively, the project explores three complementary strategies: (1) a traditional approach using statistical visual features with classical classifiers, (2) a transfer learning approach leveraging deep feature extraction with standard classifiers, and (3) a benchmark approach employing state-of-the-art dedicated detection models. By evaluating these multiple methods, this work aims to contribute valuable insights and practical solutions to the growing problem of AI-based visual misinformation.

DATASET DESCRIPTION

At the beginning of this project, a different dataset was initially planned. However, since that dataset did not include a proper description and detailed structure, it was later replaced with a more comprehensive and well-documented dataset to ensure clarity and reproducibility of the study. For this purpose, the “140k Real and Fake Faces” dataset, which is publicly available on Kaggle was used.

Data Link: https://www.kaggle.com/datasets/xhlulu/140k-real-and-fake-faces

This dataset combines two large and balanced collections:
-70,000 real face images collected from the Flickr dataset by Nvidia.
-70,000 fake face images generated using StyleGAN technology, provided by Bojan in the “1 Million Fake Faces” dataset.

All images were resized to a uniform resolution of 256×256 pixels to standardize the input for machine learning models. The dataset creator has already split the data into training, validation, and testing sets, and provided CSV files (train.csv, valid.csv, test.csv) that include image paths and corresponding labels.

large number of diverse, high-quality face images that cover various poses and lighting conditions, making it realistic and challenging for algorithms to distinguish real faces from synthetically generated ones.

In conclusion, this dataset provides a reliable and well-documented foundation for training, validating, and testing machine learning and deep learning models developed in this project to detect AI-generated faces.

FIRST APPROACH

In the first approach, different visual features were extracted from each image. These features were then used to train various classical machine learning algorithms. This method focuses on distinguishing real and fake images through traditional feature extraction and classification techniques. Table 1 shows the nine handcrafted features extracted from each image in the first approach.

Method Name	Short Description
Color Histogram	Extracts color distribution in RGB channels; useful to capture dominant colors and color diversity.
Grayscale Statistics	Computes mean and standard deviation of the grayscale image; summarizes brightness and contrast.
Edge Density	Measures the proportion of edge pixels detected by Canny edge detector; indicates texture and boundary richness.
Local Binary Pattern (LBP)	Encodes local texture by comparing each pixel to its neighbors; robust for texture classification.
Histogram of Oriented Gradients (HOG)	Describes object shape by capturing gradient orientation patterns; commonly used for detection tasks.
Gabor Features	Extracts frequency and orientation information using Gabor filters; effective for texture and pattern recognition.
Wavelet Features	Computes mean and standard deviation of wavelet decomposition coefficients (LL, LH, HL, HH); captures multi-scale information.
Image Moments	Calculates central moments (mu20, mu02, mu11); provides shape and geometric information.
Shannon Entropy	Measures the information content (randomness) of the image; higher entropy indicates more complexity.

Table 1: Feature Extraction Methods Used in the First Approach

Table 2 summarizes the supervised machine learning algorithms and their key hyperparameters employed in the first approach. These models were trained and test using handcrafted features.

Algorithm	Key Parameters / Settings	Short Description
Decision Tree	Random State: 42	Tree-based model; splits data based on feature thresholds.
Random Forest	Estimators: 200, Random State: 42	Ensemble of decision trees; improves accuracy by averaging multiple trees.
Logistic Regression	Max Iterations: 1000, C=0.5	Linear model for binary classification; estimates class probabilities.
K-Nearest Neighbors (KNN)	Neighbors: 5	Classifies based on majority label among the k closest samples in the feature space.
Multi-layer Perceptron (MLP)	Hidden Layers: (100, 50), Max Iterations: 500, Random State: 42	Feedforward neural network with two hidden layers; learns complex decision boundaries.
XGBoost	Use Label Encoder: False, Eval Metric: ‘logloss’, Random State: 42	Gradient boosting framework; robust and efficient for tabular data.
LightGBM	Random State: 42	Light Gradient Boosting Machine; faster and scalable boosting method.
CatBoost	Verbose: 0, Random State: 42	Categorical Boosting; handles categorical features automatically and avoids overfitting.

Table 2: Algorithms and Parameters Used in the First Approach

To reliably assess the models in the first approach, Stratified 3-Fold Cross-Validation was applied. This method splits the dataset into three folds, keeping the same proportion of real and fake images in each fold. In each round, two folds are used for training and one for testing. This process repeats three times, and the average result represents the model's performance. This strategy helps avoid biased results and overfitting by ensuring balanced class distribution and more robust evaluation.

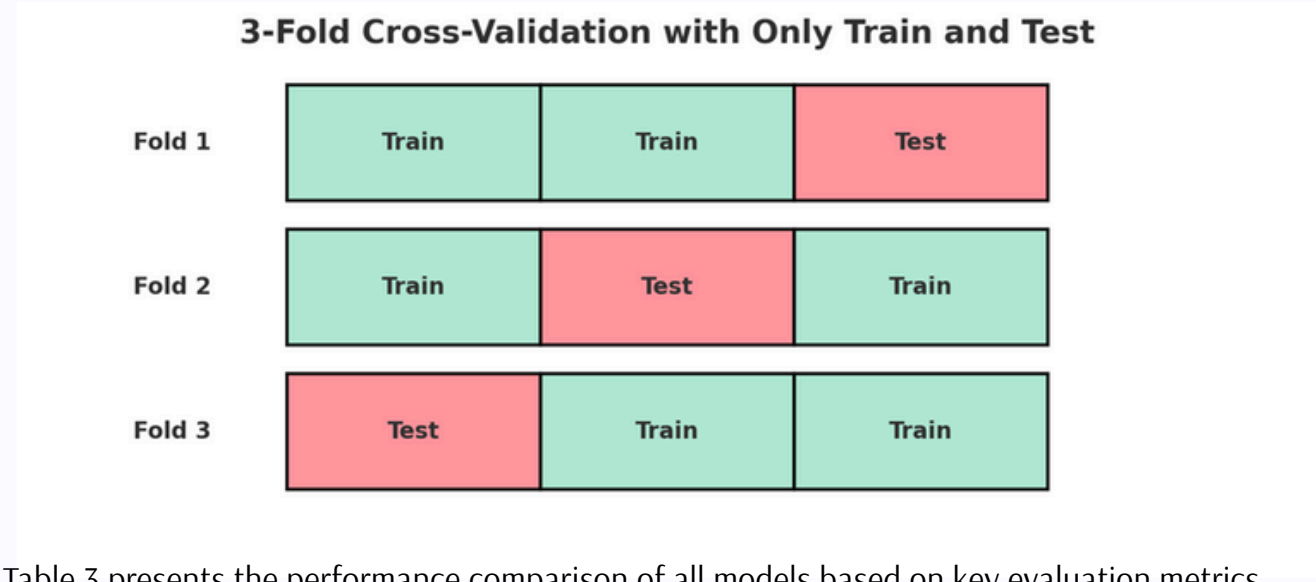


Table 3 presents the performance comparison of all models based on key evaluation metrics including accuracy, precision, recall, F1-score, and total execution time.

Models	Total-Time	Accuracy	Precision	Recall	F1-Score
KNN	121.077	0,5668	0,5359	0,9916	0,6955
DECISION TREE	58.142	0,6760	0,6724	0,6853	0,6785
RANDOM FOREST	815.595	0,8257	0,8549	0,7846	0,8185
LOGISTIC REGRESSION	68.114	0,8831	0,8811	0,8856	0,8833
MULTI-LAYER PERCEPTION	210.804	0,9411	0,9394	0,9431	0,9412
XGBOOST	10.028	0,8971	0,8977	0,8963	0,8969
LIGHTGBM	80.051	0,8674	0,8767	0,8552	0,8645
CATBOOST	417.236	0,9251	0,9262	0,9237	0,9243

Table 3: Summary of model performance across key evaluation metrics and runtime.

SECOND APPROACH

In the second phase of this project, feature extraction was performed using a transfer learning strategy based on a pre-trained deep convolutional neural network. Specifically, a ResNet50 model, pre-trained on the ImageNet dataset, was employed as a fixed feature extractor.

First, all images were resized to 256×256 pixels and normalized using the standard ImageNet mean and standard deviation values. For training data, random horizontal flipping was also applied as a data augmentation technique to increase the model's generalization ability.

Next, the fully connected (classification) layer of ResNet50 was removed, and the output of the last convolutional block was used as the feature representation for each image. This approach leverages the powerful feature extraction capability of ResNet50, which has been trained on millions of diverse images and thus captures rich and discriminative visual patterns.

Feature extraction was performed separately for the training, validation, and test sets. All extracted features were flattened into one-dimensional vectors and stored as .npy files for subsequent training and evaluation with various traditional classifiers. This method enables the use of deep learned features while avoiding the computational cost of training the entire network from scratch, which is particularly advantageous when working with limited computational resources or smaller datasets.

Link:https://docs.pytorch.org/vision/stable/models/generated/torchvision.models.resnet50.html

Table 4 summarizes the machine learning classifiers and their key hyperparameters utilized to train and evaluate the extracted deep features from ResNet50 in the second approach.

Algorithm	Key Parameters / Settings	Short Description
Decision Tree	Random State: 42	Tree-based model; splits data based on feature thresholds.
Random Forest	Estimators: 200, Random State: 42	Ensemble of decision trees; improves accuracy by averaging multiple trees.
Logistic Regression	Max Iterations: 1000, C=0.5	Linear model for binary classification; estimates class probabilities.
K-Nearest Neighbors (KNN)	Neighbors: 5	Classifies based on majority label among the k closest samples in the feature space.
Multi-layer Perceptron (MLP)	Hidden Layers: (100, 50), Max Iterations: 500, Random State: 42	Feedforward neural network with two hidden layers; learns complex decision boundaries.
XGBoost	Use Label Encoder: False, Eval Metric: ‘logloss’, Random State: 42	Gradient boosting framework; robust and efficient for tabular data.
LightGBM	Random State: 42	Light Gradient Boosting Machine; faster and scalable boosting method.
CatBoost	Verbose: 0, Random State: 42	Categorical Boosting; handles categorical features automatically and avoids overfitting.

Table 4: Algorithms and Parameters Used in the First Approach

In this approach, the data was split into three parts: training, validation, and test sets. The training and validation sets were combined and used with 3-fold cross-validation to train and validate the models. Each fold used a different part for validation and the rest for training. After cross-validation, the separate test set was used only once to check final performance. This ensures fair testing and prevents data leakage.

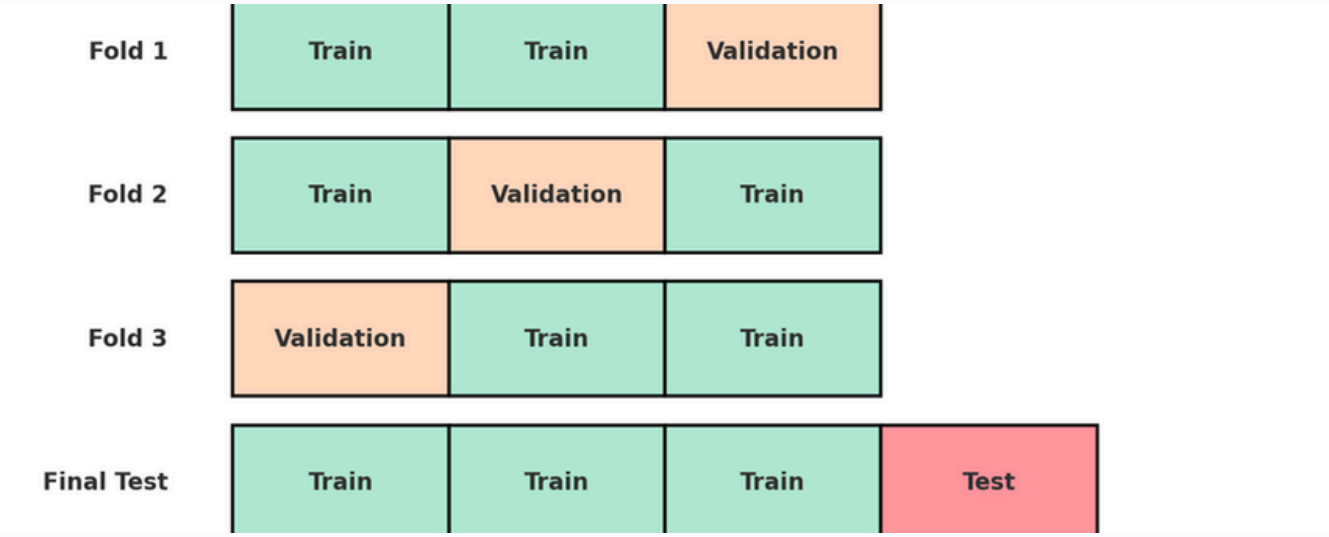


Table 5 presents the performance comparison of all models based on key evaluation metrics including accuracy, precision, recall, F1-score, and total execution time.

Models	Total-Time	Accuracy	Precision	Recall	F1-Score
KNN	35.489	0,7611	0,9279	0,5661	0,7032
DECISION TREE	91.591	0,7056	0,7087	0,6979	0,7033
RANDOM FOREST	199.082	0,8391	0,8419	0,8350	0,8384
LOGISTIC REGRESSION	12.281	0,9271	0,9297	0,9240	0,9268
MULTI-LAYER PERCEPTION	5.299	0,9544	0,9450	0,9648	0,9548
XGBOOST	23.492	0,9105	0,9143	0,9059	0,9101
LIGHTGBM	15.723	0,8882	0,8936	0,8814	0,8875
CATBOOST	255.352	0,9298	0,9348	0,9242	0,9295

Table 5: Summary of model performance across key evaluation metrics and runtime.

THIRD APPROACH

In the third approach, additional reference models were integrated into the study to provide a stronger baseline and to improve the comparison with the first two methods. For this purpose, advanced synthetic image detection models were utilized through the public implementation available at the SIDBench GitHub repository, which offers a standardized benchmark and pre-trained detectors specifically designed for identifying AI-generated images.

SIDBench repository link: https://github.com/mever-team/sidbench.

In this stage, two well-known models — UnivFD and DIMD — were selected because they are capable of detecting subtle artifacts and inconsistencies typically found in images created by GANs, making them highly effective for synthetic image detection tasks. Additionally, other models provided in the repository, such as CNNDetect, GramNet, and LGrad, were also tested but did not yield consistent or reasonable results within the scope of this project.

All experiments were run with NVIDIA CUDA acceleration on a system equipped with an RTX 2060 GPU, which ensured efficient computation and faster inference times.

A detailed comparison of the evaluation results for these reference models is presented in Table 6, alongside the results from the previous approaches.

MODELS	Total-Time	Accuracy	Precision	Recall	F1-Score
UnivFD	11588 (3h)	0,8122	0,9692	0,3492	0,5134
DIMD	61435 (17h)	0,8923	0,9971	0,3149	0,4786

Table 6: Performance metrics and runtime for UnivFD and DIMD models in the third approach.

CONCLUSION

In conclusion, this study shows that the second approach, which uses features extracted by ResNet50 combined with classical classifiers, is the most balanced and effective method for detecting AI-generated images. This approach achieved the highest overall accuracy and F1-scores. For example, the MLP classifier with deep features reached about 0.954 accuracy, slightly higher than the same model using traditional features in the first approach, which scored around 0.941. Other classifiers also performed better with ResNet50 features — for instance, logistic regression improved from about 0.883 to over 0.927 accuracy. This consistent improvement, together with short training times, makes the second approach the most practical in terms of both performance and efficiency. The first approach, based on manually designed visual features, still produced competitive results, especially with models like CatBoost and XGBoost, and can be a good option when computational resources are limited or model transparency is needed. The third approach, which applied specialized detectors such as UnivFD and DIMD, showed very high precision values (up to 0.997) but lower recall values (around 0.300–0.350) and required significantly longer processing times (for example, DIMD needed about 17 hours). Overall, using ResNet50 for feature extraction and training an MLP or CatBoost provides the best trade-off between accuracy, speed, and practicality, while the other approaches remain useful alternatives depending on specific project requirements and constraints.

FUTURE STEPS

- Expand and diversify the dataset by including images generated by newer GANs, diffusion models, or various deepfake tools.
- Evaluate the models on entirely different datasets to test their robustness and generalization in real-world conditions.
- Integrate automated hyperparameter optimization tools, such as Bayesian optimization or grid search frameworks, to systematically find the best configuration for each model.
- Develop an interactive user interface or dashboard to make the detection tool more accessible for non-technical users.
- Adapt the detection pipeline to handle AI-generated videos by analyzing sequences of frames instead of single images.
- Upgrade to more powerful GPUs or TPUs to significantly reduce training and inference times for large-scale experiments.